LearnQuest LMS — Source Code Documentation

AI-powered Adaptive Learning Management System · BSIT Capstone · Kapitolyo High School (Grade 11–12 STEM, Science / Chemistry)

Snapshot of branch `main` at commit `cfb9ea8` · Generated 2026-10-11 · Test suite at snapshot: **315 passed (2134 assertions)**

> **Purpose of this document.** This is a context brief for AI coding agents and new developers. It describes what the code *actually does* today: the architecture, data model, routes, services, AI pipeline, BKT mastery tracking, conventions, and known gaps. Read Section 1 (non-negotiable rules) before you change anything. When this document and the code disagree, the code wins. Re-check file paths before you rely on them.

Contents

1. Non-negotiable project rules
2. Technology stack
3. Repository layout
4. Architecture overview
5. Domain vocabulary
6. Database schema
7. Authentication, roles & security
8. HTTP routes reference
9. Service layer catalog
10. AI subsystem (Routeway / Llama)
11. Bayesian Knowledge Tracing & adaptive learning
12. End-to-end flows
13. Analytics, insights & notifications
14. Frontend
15. Configuration reference
16. Testing
17. Coding conventions
18. Known gaps & gotchas
19. Quick task recipes

1. Non-negotiable project rules

These come from `CLAUDE.md` (the project specification) and the research design. Code that breaks them is wrong even if it works.

1. **BKT estimates mastery; Llama only generates content.** Never have the LLM calculate, guess, or overwrite mastery. Mastery comes only from `BayesianKnowledgeTracingService`.
2. **Raw quiz score is not mastery.** 10/20 = 50% is an assessment score. BKT mastery is a probability per competency computed from the response sequence. They are stored and displayed separately.
3. **Mastery is per competency,** keyed by (student, competency), not per quiz or per subject. Lesson- and subject-level figures are averages that are computed when displayed.
4. **Professors do not write quizzes.** Quizzes are AI-generated from the uploaded lesson PDF. Professors configure settings, review (approve or reject) questions, and can correct a question's competency or difficulty. Do not add a manual quiz editor.
5. **Credentials stay server-side.** The browser never calls Routeway. Keys live in `.env`, are read through `config/services.php`, and are never sent to JS, Blade, or git.
6. **Validate all LLM output before saving or showing it.** Every AI service parses JSON, validates each field against the database (competency IDs, difficulty enum, correct answer is one of the choices), drops invalid items, and throws `InvalidAiResponseException` if nothing valid remains.
7. **Thin controllers, logic in services.** Thresholds and limits live in `config/bkt.php` and `config/quiz.php`. Do not hard-code them.
8. **Preserve the existing UI.** The HTML/CSS/JS frontend under `frontend/` is the product UI. Migrate it in small steps and do not redesign it.
9. **Stay in scope.** No Random Forest, DLKT or MDP for mastery, and no vector database unless it is clearly required. Simple, secure, testable.
10. **Never send student names to the LLM** in professor-facing features. Only aggregate figures are sent.

2. Technology stack

Layer	Technology
Backend	PHP 8.2+, Laravel 12 (web routes only, session auth, no `routes/api.php`)
Database	MySQL in development per spec (`learnquest_lms`). Note: `.env.example` defaults to `sqlite`. Tests use SQLite `:memory:`.
Session / cache / queue	`database` drivers. No queued jobs exist; all AI calls are synchronous.
Storage	Default Laravel disk (`FILESYSTEM_DISK`): local in dev, S3-compatible bucket (Laravel Cloud) in production via `league/flysystem-aws-s3-v3`
Documents	`smalot/pdfparser` (PDF text extraction), `phpoffice/phppresentation` (writes generated `.pptx` slide decks)
Auth extras	`laravel/socialite` (Google sign-in), email OTP (custom `OtpService`)
LLM	Routeway OpenAI-compatible API (`https://api.routeway.ai/v1`). Model chain: `llama-3.3-70b-instruct` → `deepseek-v4-flash` → `deepseek-v4-flash:free`. Uses a Guzzle client directly.
Frontend	Static HTML + vanilla JS + CSS in `frontend/`, served through Laravel controllers behind auth. Font Awesome and Inter come from CDNs. Vite/Tailwind 4 are scaffolded but the product UI does not use them.
Dev tooling	PHPUnit 11, Laravel Pint, Pail, Boost (MCP), Sail. `composer run dev` starts server + queue + logs + vite.

3. Repository layout

```
learnquest-lms/
├── app/
│   ├── Exceptions/AI/        LlamaApiException, InvalidAiResponseException, NoSourceContentException
│   ├── Http/
│   │   ├── Controllers/      Domain controllers (classes, posts, modules, subjects, quiz, settings, auth)
│   │   │   └── AI/           Summary, QuizGeneration, QuestAi, ProfessorQuestAi, Insight, ClassInsight,
│   │   │                     CompetencySuggestion, SlideDeck
│   │   ├── Middleware/       EnsureStudentRole, EnsureProfessorRole, EnforceIdleLock
│   │   └── Requests/         Form requests (validation) for every write endpoint
│   ├── Mail/OtpCodeMail.php
│   ├── Models/               Eloquent models (see §6)
│   ├── Notifications/        Database notifications ("System Alerts")
│   └── Services/
│       ├── AI/               LlamaService (HTTP client), PromptService (all prompts), feature services
│       ├── Bkt/              BayesianKnowledgeTracingService (pure math)
│       ├── Learning/         AdaptiveLearning, AdaptiveQuiz, StudentAnalytics, ProfessorAnalytics
│       ├── Quiz/             QuizAttempt (grading → BKT), QuizSettings, QuizFeedback, QuizTraining
│       ├── Documents/        PdfTextExtractor, LessonContent (retrieval), StoredFile, PresentationBuilder
│       ├── Auth/             OtpService, LoginChallengeService, IdleLockService
│       ├── Account/          AccountDeletionService
│       └── Notifications/    AlertService
├── bootstrap/app.php         Middleware aliases: student.role, professor.role; EnforceIdleLock on web group
├── config/                   bkt.php, quiz.php, otp.php, services.php (routeway, google), logging.php ('ai'
channel)
├── database/migrations/      Schema (see §6). database/seeders/ClassSeeder.php
├── docs/                     PROJECT_CONTEXT.md, capstone paper (Ch. 1–2 PDF), this document
├── frontend/                 THE PRODUCT UI (served via FrontendShellController)
│   ├── pages/{student,professor,auth,admin}/*.html
│   ├── components/navbar/    Shell navbar (hosts the page iframe), pdf-modal/
│   └── assets/{css,js,images}/  common/*.js shared scripts; pages/{role}/*.js per page
├── resources/views/          Blade: landing, auth/*, app/shell.blade.php, emails/otp*
├── routes/web.php            ALL routes (JSON endpoints included)
├── tests/{Unit,Feature}/     PHPUnit (see §16)
├── CLAUDE.md                 Project spec for agents (authoritative intent)
└── AGENTS.md                 Laravel Boost guidelines (coding conventions)
```

4. Architecture overview

4.1 Request path

```
Browser (frontend/pages/*.html inside the shell iframe, vanilla JS fetch + X-XSRF-TOKEN)
   ↓  session cookie
routes/web.php  → middleware: auth, student.role | professor.role, EnforceIdleLock, throttle:*
   ↓
Controller (thin: authorize ownership/enrollment, validate via FormRequest, call service, return JSON)
   ↓
Service layer  ── Bkt / Learning / Quiz  (deterministic rules + math)
               └─ AI feature service → PromptService (messages) → LlamaService (HTTP) → Routeway
   ↓
Eloquent models / MySQL · Storage disk (PDFs, avatars, decks) · 'ai' log channel (storage/logs/ai.log)
```

4.2 Layering rules seen in the code

- `LlamaService` handles auth, retries, the model chain, parallel requests, and logging. It contains **no prompts or business logic**.
- `PromptService` builds **every** prompt (one `*Messages()` method per feature). Controllers contain no prompt text.
- Feature services (`QuizGenerationService`, `SummarizationService`, …) gather context, call Llama, and run their own `parseAndValidate()` before they persist anything.
- Analytics and insight services do not call BKT to *change* mastery. They read `student_mastery` or replay `quiz_answers` through the BKT math.
- When an AI-backed feature has no source data, it falls back to **rule-based output** built from the same BKT data (insights, class insights), so the UI still works when the LLM is down.
- Authorization uses private helper methods in each controller (`authorizeEnrolled`, `authorizeOwner`, `authorizeManager`, `authorizePostBelongsToClass`, …), plus `Subject::isManagedBy()`. There are no Policy classes. Out-of-scope resources return **404**, not 403.

5. Domain vocabulary

The code's names differ from the generic LMS terms in `CLAUDE.md`. Map them like this:

Concept	Code / table	Notes
Subject (curriculum container)	`Subject` / `subjects`	Owned by a professor (`owner_id`), shared with **collaborators** (`subject_collaborators`), who act as co-owners. Holds modules, competencies and sections.
Class / Section	`ClassRoom` / `classes`	A teachable section with a join `code`. `subject_id` links it to a Subject (relation **`parentSubject()`**, because the `subject` column is a legacy free-text label). Can be archived.
Enrollment	`ClassEnrollment` / `class_enrollments`	Students join with a class code.
Module (learning material)	`Module` / `modules`	A PDF uploaded to a Subject, targeted at sections (`module_sections`), and tagged with a quarter.
Lesson	`ClassPost` with `type='lesson'`	Targeting a module creates one lesson post per class, with a *copy* of the PDF. **All AI artifacts (summary, quiz, settings) attach to the ClassPost**, not to the Module.
Announcement	`ClassPost` with `type='announcement'`	Optional checklist and PDF attachment.
Competency	`Competency` / `competencies`	Defined per Subject (unique name). Professors create them, optionally from AI suggestions. Every quiz question maps to exactly one.

Mastery	`StudentMastery` / `student_mastery`	BKT record per (student, competency).
QuestAI	`AiConversation`, `AiMessage`	Student tutor chat, plus the professor "QuestAI Coach" (same tables, `professor_id` set).
Quiz & AI Setup / "Quiz Studio"	`QuizStudioController`	Professor page for quiz settings, question review, top-up and calibration metrics.
System Alerts	Laravel database notifications	A notification drawer shared by all roles.

6. Database schema

Columns below come from the migrations. `id` and timestamps are omitted. FK = foreign key.

Table	Key columns
`users`	name, email, password, email_verified_at, **role** enum(student, professor, admin), google_id (unique, nullable), avatar_path, notification_preferences (json), general_preferences (json; start page, tour flags), idle_lock_minutes (0 = off), otp_on_login (bool)
`subjects`	owner_id→users, name
`subject_collaborators`	subject_id, user_id, invited_by · unique(subject_id, user_id)
`classes`	professor_id→users, subject_id→subjects (nullable), name, section, subject (legacy string), room, code (unique), archived_at
`class_enrollments`	class_id, student_id · unique pair
`modules`	subject_id, uploaded_by, title, description, quarter, file_path, file_name, file_size
`module_sections`	module_id, class_id · unique pair (which sections a module targets)
`class_posts`	class_id, module_id (nullable), author_id, type (announcement|lesson), quarter, title, body, checklist (json), attachment_path, attachment_name, edited
`lesson_summaries`	class_post_id (unique), overview, key_points (json), model, generated_at
`competencies`	subject_id, name, description · unique(subject_id, name)
`quizzes`	class_post_id, model, generated_at, **archived_at**. At most one active quiz per post. Regeneration archives the old one.
`quiz_questions`	quiz_id, competency_id (current, teacher-correctable), ai_competency_id (AI's original tag), question_text, choices (json), correct_answer, explanation, difficulty (easy|medium|hard), order_index
`quiz_question_reviews`	quiz_question_id (unique), professor_id, verdict (approved|rejected), reason, comment, ai_difficulty, teacher_difficulty
`quiz_settings`	class_post_id (unique), professor_id, question_count, time_limit_minutes (null = none), difficulty_mix json {easy,medium,hard %}, max_attempts (null = unlimited), shuffle_questions, shuffle_choices, show_answers, adaptive
`quiz_attempts`	quiz_id, student_id, started_at, deadline_at, **question_ids** (json; the subset served), **mastery_level** (low|developing|high at serve time), submitted_at, score_correct, score_total, status (in_progress|submitted|late)
`quiz_answers`	quiz_attempt_id, quiz_question_id, **competency_id**, selected_index (null = skipped), is_correct (null = skipped), answered_at · unique(attempt, question). **This is the BKT observation log.**
`student_mastery`	student_id, competency_id (unique pair), initial_mastery, current_mastery, p_l0, p_t, p_g, p_s (decimal 5,4), observations_count, correct_count, incorrect_count, last_response, last_updated_at
`quiz_feedback`	quiz_attempt_id (unique), quiz_id, student_id, rating, difficulty, comment. Students rate a quiz after submitting.
`ai_conversations`	student_id (nullable), professor_id (nullable), class_id, title, last_message_at
`ai_messages`	ai_conversation_id, role (user|assistant), content, model, source_post_ids (json; grounding sources), feedback (thumbs ±1)
`otp_codes`	email, purpose, code_hash, attempts, expires_at, last_sent_at · unique(email, purpose)
`notifications`	Standard Laravel database notifications (uuid id, morphs notifiable, data, read_at)

Laravel defaults also exist: password_reset_tokens, sessions, cache, jobs. Two migrations add `role` to users (2024_01_15 and 2026_07_08) with different enums. Check the live schema if you depend on the `admin` value.

6.1 Key Eloquent relationships

```
User ── classes() (as professor) · enrolledClasses() · ownedSubjects() · collaboratingSubjects()
     ── quizAttempts() · masteryRecords()
Subject ── owner · collaborators · sections() → ClassRoom · modules · competencies · isManagedBy(User)
ClassRoom ── professor · parentSubject · students · posts
ClassPost ── classRoom · author · module · summary (LessonSummary) · quiz (active Quiz) · quizSetting
Quiz ── classPost · questions · activeQuestions() (excludes teacher-rejected) · attempts · feedback
QuizQuestion ── quiz · competency · aiCompetency · answers · review
QuizAttempt ── quiz · student · answers · feedback
StudentMastery ── student · competency
```

7. Authentication, roles & security

- **Roles:** `student` and `professor` are used. The middleware aliases `student.role` and `professor.role` redirect anyone else to `/login`. `admin` exists in an enum, but **there are no admin routes or controllers** (§18).
- **Registration** (`AuthController@register`): the user picks a role and must verify the account with an emailed **OTP** before signing in.
- **Login:** email + password, with remember-me. An unverified account is sent to OTP verification. If the user enabled `otp_on_login`, `LoginChallengeService` holds the sign-in until an emailed code is entered.
- **Google sign-in** (`GoogleAuthController`): enabled only when `GOOGLE_CLIENT_ID` is set. A new Google user must choose a role on `/auth/google/role`.
- **Password reset:** through the OTP flow (`/forgot-password` → `/verify-otp` → `/reset-password/{token}`).
- **OTP** (`OtpService`, `config/otp.php`): 6 digits, hashed, valid for 10 minutes, at most 3 attempts, 60 s resend cooldown, one row per (email, purpose).
- **Idle lock** (`IdleLockService` + `EnforceIdleLock` web middleware): after `idle_lock_minutes` without activity, the session is locked until the user enters an emailed code. The state is stored per session. The frontend sends a heartbeat to `/session/heartbeat`.
- **Throttling:** auth, OTP, AI, and class-join endpoints use `throttle:N,1`. Quiz generation is limited to 6/min, slide decks to 5/min, and chat to 20/min.
- **Uploads:** module and post attachments must be `mimes:pdf`, max 10 MB. Files are served only through authorized controller endpoints and never from a public path.
- **Frontend assets** are served by `FrontendShellController`, which uses `realpath` containment and rejects `..` and paths outside `frontend/assets`.
- **Quiz answer secrecy:** `QuizGenerationController@show` never returns `correct_answer`, `explanation`, `competency_id` or `difficulty`. Answers are revealed only in the graded review after submission, depending on the `show_answers` setting.
- **Account deletion** (`AccountDeletionService`): deletes the user's own data. A subject that other people still use is transferred to a new owner, and content the user contributed to others' subjects or classes stays in place.

8. HTTP routes reference

Everything is in `routes/web.php`. Paths that serve data return JSON. The class-scoped routes use bindings `{class}` =ClassRoom, `{post}` =ClassPost, `{attempt}`, `{question}`, `{subject}`, `{module}`, `{competency}`.

8.1 Public / auth

Method & path	Handler
GET /	landing view
GET|POST /login · POST /register · POST /logout	AuthController
GET /auth/google · /auth/google/callback · GET|POST /auth/google/role	GoogleAuthController
GET|POST /forgot-password · GET /reset-password/{token} · POST /reset-password	AuthController
GET|POST /verify-otp · POST /verify-otp/resend	OtpController

8.2 Any authenticated user

Method & path	Purpose
GET /assets/{path} · /components/navbar/navbar.{css,js}	Serve frontend assets (FrontendShellController)
GET|POST /session-locked · POST /session-locked/resend · POST /session/lock · GET /session/status · POST /session/heartbeat	Idle lock
GET /notifications · POST /notifications/read-all · POST /notifications/{id}/read · DELETE /notifications	System Alerts
GET /avatars/{user}	Another user's avatar
/settings: GET / · PUT profile, notifications, general, security, password · DELETE general, account · GET|POST|DELETE avatar	SettingsController
GET /pages/student/{page}.html · /pages/professor/{page}.html	Role-gated static pages from `frontend/pages/`

8.3 Student (`/student`, middleware auth + student.role)

Method & path	Handler → purpose
GET /dashboard	FrontendShellController@showShell (the SPA-like shell)
GET /analytics	StudentDashboardController@data → StudentAnalyticsService
GET /classes · GET /classes/lookup/{code} · POST /classes/join · GET /classes/{class}	StudentClassController
GET /classes/{class}/mastery · /classes/{class}/posts/{post}/mastery	StudentMasteryController (BKT per competency)
GET /classes/{class}/insights	InsightController → InsightService (AI study advice)
GET /classes/{class}/posts · /posts/{post}/attachment	StudentClassPostController
GET /classes/{class}/posts/{post}/summary	SummaryController@show (generated on first request, then cached)
GET /classes/{class}/posts/{post}/quiz	QuizGenerationController@show (generates on first request, opens or resumes an attempt)
POST /classes/{class}/posts/{post}/quiz/attempts	QuizAttemptController@store → QuizAttemptService (grading + BKT)

POST …/quiz/attempts/{attempt}/feedback	QuizFeedbackController@store
/questai: GET context · GET conversations · GET conversations/{c} · POST messages · POST messages/{m}/feedback	QuestAiController → QuestAiService

8.4 Professor (`/professor`, middleware auth + professor.role)

Method & path	Handler → purpose
GET /dashboard · GET /analytics · GET /analytics/insights	Shell · ProfessorAnalyticsService · ClassInsightService
/questai: context, conversations, messages, feedback · POST /decks · GET /decks/{deck}/download	ProfessorQuestAiController (Coach) · SlideDeckController
Classes: GET|POST /classes · GET /classes/archived · GET|PUT|DELETE /classes/{class} · POST regenerate-code, archive, restore · GET students · DELETE students/{student}	ProfessorClassController
Posts: GET|POST /classes/{class}/posts · PUT|DELETE posts/{post} · GET posts/{post}/attachment	ClassPostController (multipart updates use POST with `_method=PUT`)
GET …/posts/{post}/quiz/feedback · POST …/quiz/regenerate	QuizGenerationController (feedback summary; regenerate from feedback)
Quiz Studio: GET /quiz-studio/lessons · /quiz-studio/training · …/quiz/studio · PUT …/quiz/settings · POST …/quiz/generate · POST …/quiz/top-up · PUT|DELETE …/questions/{q}/review · PUT …/questions/{q}/competency	QuizStudioController
Subjects: GET|POST /subjects · GET|PUT /subjects/{subject} · POST sections · POST|DELETE collaborators	SubjectController, SubjectSectionController, SubjectCollaboratorController
Modules: GET|POST /subjects/{s}/modules · PUT|DELETE modules/{m} · PUT modules/{m}/sections · POST modules/{m}/change-requests · GET modules/{m}/attachment	ModuleController
Competencies: GET|POST /subjects/{s}/competencies · POST competencies/suggest · PUT|DELETE competencies/{c}	CompetencyController, CompetencySuggestionController

9. Service layer catalog

Service	Responsibility
AI (app/Services/AI)	
LlamaService	Low-level Routeway client. `chat()` (sequential with retries and the model chain) and `chatConcurrent()` (parallel batch requests with a per-request fallback and an optional `$parse` validator). Returns `{content, model}`.
PromptService	Builds every prompt: lessonSummary, competencySuggestion, quizGeneration, studentInsight, classInsight, questAiTutor, professorCoach, slideDeck.
SummarizationService	PDF → text (12k char cap) → JSON `{overview, key_points[≤6]}` → validate → `LessonSummary::updateOrCreate`
QuizGenerationService	`generateForPost`, `regenerateForPost` (archive + feedback-informed), `topUp` (replace rejected questions). Uses parallel batches and strict per-question validation.
QuestAiService	Student tutor. Grounded in lesson summaries and retrieved PDF chunks, with the student's BKT profile as context. Stores `source_post_ids`.
ProfessorQuestAiService	Professor "Coach", grounded in the professor's materials and class-level aggregate mastery (no student names).
InsightService	Per-student study advice from BKT weaknesses and strengths. Falls back to rule-based output and is cached.
ClassInsightService	Strengths-vs-weaknesses analysis for the professor dashboard. Falls back to rule-based output.
CompetencySuggestionService	Proposes competencies from a subject's module text. **Saves nothing**; the professor adds the ones they choose.
SlideDeckGenerationService / SlideDeckFocusService	Outline JSON from a reading, in parallel parts of ≤5 slides, optionally focused on the class's weak competencies → PresentationBuilderService writes the .pptx
Mastery & learning	
Bkt\BayesianKnowledgeTracingService	Pure BKT: `initializeMastery`, `getMastery`, `updateAfterCorrectResponse`, `updateAfterIncorrectResponse`, `nextMastery` (math), `replay` (rebuild trajectory), `averageMasteryForCompetencies`
Learning\AdaptiveLearningService	`classify(mastery)` → low/developing/high. `competencyProfile`, `weaknesses`, `strengths`. Unassessed competencies are never reported as weaknesses or strengths.
Learning\AdaptiveQuizService	`masteryFor` (average over assessed competencies only), `mixFor` (shift the difficulty mix), `poolAllocation` (bank size), `select` (pick questions, filling gaps from neighbouring difficulties)
Learning\StudentAnalyticsService / ProfessorAnalyticsService	Dashboard data. Mastery over time is **reconstructed by replaying quiz_answers through BKT**, so it can be filtered by quarter, date or section.
Quiz	
Quiz\QuizAttemptService	`submit()` in a DB transaction: grades the served questions, writes `quiz_answers`, updates BKT for each answered question, and returns score, review, mastery deltas, and lesson/subject mastery before and after.
Quiz\QuizSettingsService	Per-lesson settings. Inherits the professor's last saved settings, then the config defaults. `allocate(count, mix)`, `attemptsUsed`.
Quiz\QuizFeedbackService	Student quiz feedback: summary, breakdown, and `promptContext` for regeneration.
Quiz\QuizTrainingService	"Training" through in-context learning (no fine-tuning). Teacher reviews and real student correct-rates become prompt guidance per subject, plus calibration metrics (`difficulty_targets`, tolerance).

Documents / other	
Documents\PdfTextExtractorService	PDF → plain text. Does not OCR scanned PDFs.
Documents\LessonContentService	Keyword-overlap retrieval (no vector DB). Extracted text is cached forever per post and file hash. Provides chunk, sentences, sections, retrieve, lessonSummaries, recentClassPosts.
Documents\StoredFile	`withLocalPath(path, fn)` : gives libraries a real file path on both local and cloud disks.
Notifications\AlertService	Decides who gets each System Alert: post published, student joined, module change request, quiz feedback, mastery level changes (falling to low alerts the student and professor; reaching high alerts only the student). Failures are logged and never thrown.
Auth\OtpService, LoginChallengeService, IdleLockService; Account\AccountDeletionService	See §7

10. AI subsystem (Routeway / Llama)

10.1 LlamaService model chain

```
chat(messages, options)
  1. primary   = services.routeway.model           (llama-3.3-70b-instruct)  — retries on 502/503/504 or
                                                    connection errors, exponential backoff (500ms × 2^n)
  2. fallback  = services.routeway.fallback_model   (deepseek-v4-flash)
  3. last resort = services.routeway.last_resort_model (deepseek-v4-flash:free) — no retries by default
  → returns ['content' => string, 'model' => string]   (store 'model' on generated rows)
  → throws LlamaApiException when every model fails, or when LLAMA_API_KEY is missing
```

- Per-call limit options (stripped before the API body): `timeout`, `max_retries`, `fallback_timeout`, `fallback_max_retries`, `last_resort_timeout`, `last_resort_max_retries`, `fallback_only`. Any other option (e.g. `max_tokens`, `temperature`) is passed to the API. The default temperature is 0.4.
- Per-feature limits live in `config/services.php` under `routeway.insights`, `questai_coach`, `quiz`, `competency_suggestions` and `slide_deck`. They are sized so the worst case (every model timing out) stays under PHP's 120 s web limit.
- `chatConcurrent()` uses Guzzle promises. A request that fails or is rejected by `$parse` is retried on the next model in the chain.
- All logs go to the `ai` channel (`storage/logs/ai.log`, or `LOG_AI_STACK=stderr` in the cloud). Each entry has a request_id, model, duration, token usage, and a 1500-char preview. The API key is never logged.

10.2 Validation pattern (copy this for new AI features)

```
raw = llama->chat(prompts->xxxMessages(...))['content']
decoded = json_decode(raw)  ⟶ if that fails, extract the outermost {...} (models sometimes wrap JSON in
prose/fences)
for each item: validate types, non-empty strings, enums, foreign IDs against DB-allowed sets
keep valid items, drop invalid ones; if none survive → throw InvalidAiResponseException (log raw ≤2000 chars)
persist only validated data, inside a DB transaction
```

10.3 Quiz question validation rules (`QuizGenerationService::validateQuestion`)

- `question`: a non-empty string. `choices`: an array of non-empty strings, count within `quiz.min_choices`–`max_choices` (2–6), no duplicates.
- `correct_answer`: must match one of the choices exactly (after trimming).
- `competency_id`: an int (or digit string) that belongs to the subject's competencies.
- `difficulty` ∈ {easy, medium, hard}. `explanation`: a non-empty string.
- Exact-duplicate question text (normalized) is dropped, both within a batch and against existing questions in a top-up.

10.4 Grounding

- Summary and quiz: text extracted from the lesson post's PDF, capped at **12,000 characters per request**. Large quizzes are split into batches (`quiz.questions_per_batch`, default 6), each covering a different section of the lesson (`LessonContentService::sections`), and the batches run in parallel.
- QuestAI: lesson summaries plus the top chunks retrieved by keyword overlap with the question (`LessonContentService::retrieve`). Source post IDs are stored on the reply.
- Quiz prompts also include: QuizTrainingService guidance (teacher reviews and calibration), student feedback (on regeneration), exact difficulty counts, and the allowed competency list with IDs.

10.5 Error handling contract

Controllers catch `LlamaApiException`, `InvalidAiResponseException`, `NoSourceContentException` and `RuntimeException` (precondition failures such as "no competencies defined" or "no attachment"). They return a user-

friendly JSON `message` with a suitable 4xx/5xx status, log the details to the `ai` channel, and never include stack traces.

11. Bayesian Knowledge Tracing & adaptive learning

11.1 The update (`BayesianKnowledgeTracingService::nextMastery`)

```
correct:   posterior = P(L)·(1−P(S)) / [ P(L)·(1−P(S)) + (1−P(L))·P(G) ]
incorrect: posterior = P(L)·P(S)     / [ P(L)·P(S)     + (1−P(L))·(1−P(G)) ]
learning:  P(L)next  = posterior + (1 − posterior)·P(T)          → clamped to [0,1]
```

Defaults (`config/bkt.php`, overridable in env): **P(L0)=0.30, P(T)=0.15, P(G)=0.25, P(S)=0.10**. These are copied into each `student_mastery` row when it is created, so later config changes do not alter existing records. One observation is recorded per answered question. Skipped questions do not update BKT.

11.2 Levels & thresholds

Level	Rule (config/bkt.php)	Adaptive effect
low	mastery < `mastery_threshold_low` (0.40)	Quiz mix shifted toward easy by `quiz.adaptive_shift` (20 pts, taken from hard first, then medium). Alerts the student and professor.
developing	between the thresholds	The teacher's difficulty mix is used unchanged
high	mastery ≥ `mastery_threshold_high` (0.75)	Mix shifted toward hard (taken from easy first, then medium)

The thresholds are **configurable working values, not research-fixed ones**. Say so in UI and docs. A student with no observations on the tested competencies gets the teacher's mix (the 0.30 prior is not treated as "low").

11.3 History & analytics

`student_mastery` holds only the current state. The history is rebuilt by ordering a student's `quiz_answers` per competency and calling `replay(pL0, params, responses)`. Dashboards use this method, so filtered charts are real BKT values and never quiz percentages.

12. End-to-end flows

12.1 Material → lesson

1. A professor creates a Subject (or is invited as a collaborator), adds Sections (classes), and defines Competencies, optionally from AI suggestions.
2. The professor uploads a Module (PDF, ≤10 MB, with a quarter) and targets sections. `ModuleController` creates a `ClassPost(type=lesson)` in each *newly* targeted class with a **copy** of the PDF. Re-saving never duplicates posts, and unchecking a class does not remove an existing post. Enrolled students get an alert.
3. Collaborators who do not own a module can submit *change requests* instead of editing it directly.

12.2 Summary

A student opens a lesson → `GET …/summary`. If a `LessonSummary` exists it is returned. Otherwise `SummarizationService` generates one, validates it, saves it, and returns it.

12.3 Quiz generation (lazy or from the Quiz Studio)

1. Preconditions: the post has an attachment, the class is linked to a Subject, and the Subject has competencies. Otherwise a `RuntimeException` is thrown with a readable message.
2. `QuizSettingsService::for(post)` → `AdaptiveQuizService::poolAllocation`. For adaptive quizzes, the bank holds enough questions per difficulty to serve every level its full mix, so it is larger than `question_count`.
3. The prompt is built with competency IDs, difficulty counts, training guidance, and (on regeneration) feedback context. Batches are generated in parallel, then validated and de-duplicated.
4. The Quiz and its QuizQuestions are saved in one transaction. `ai_competency_id` keeps the AI's original tag.

12.4 Serving a quiz to a student (`QuizGenerationController@show`)

1. Enrollment and post ownership are checked, then the `max_attempts` limit (403 when exhausted).
2. An existing `in_progress` attempt is resumed, so a page reload keeps the same questions and timer. Otherwise: `masteryFor` → `mixFor` → `select`, and a new attempt is created with `question_ids`, `mastery_level` and `deadline_at`.
3. The questions are returned without answers or metadata, shuffled according to the settings.

12.5 Submission → BKT

1. `QuizAttemptService::submit` grades only the served, non-rejected questions. Answers submitted after the deadline plus `time_limit_grace_seconds` (60) are graded but the attempt is marked `late`, so BKT still gets the evidence.
2. Each answered question is graded by exact trimmed-text comparison to `correct_answer`, written to `quiz_answers`, and fed to BKT for its competency.
3. The response contains the raw score, a per-question review, mastery deltas per competency (with a `first_assessment` flag), and lesson and subject mastery before and after.
4. The controller calls `AlertService::quizGraded` to send level-change alerts. The student can then submit quiz feedback.

12.6 Teacher quality loop

- **Review:** the professor approves or rejects questions (with reasons) and can correct difficulty or competency. Rejected questions disappear from `activeQuestions()` but are kept for history.
- **Top-up:** generates replacements for the difficulties that are now missing, avoiding existing question texts.
- **Regenerate:** requires ≥ `quiz.min_feedback_for_regeneration` student feedback entries. The old quiz is archived and its attempts and mastery remain.
- **Training metrics:** each question's actual % correct is compared with `difficulty_targets` (0.80/0.60/0.40 ± 0.15, after ≥5 responses) to flag mislabeled difficulty. The results feed future prompts.

13. Analytics, insights & notifications

- **Student dashboard** (`/student/analytics`): mastery per competency (radar chart), trends, quiz performance, filterable by quarter and date range. AI Insights per class are available at `/student/classes/{class}/insights` .
- **Professor dashboard** (`/professor/analytics` , `ProfessorAnalyticsRequest`): covers the sections the professor owns or collaborates on. Filterable by section, subject, quarter and date. Shows class mastery per competency, students who may need intervention (low mastery), and the AI strengths-vs-weaknesses analysis at `/analytics/insights` .
- **Notifications** (`app/Notifications`): ClassPostPublished, StudentJoinedClass, ModuleChangeRequested, QuizFeedbackReceived, MasteryLevelChanged, StudentNeedsIntervention, SystemAlert. Each user's role-specific preferences (`User::ALERT_PREFERENCES`) control which ones they receive.
- The radar charts are visualizations only. The source of truth is `student_mastery` and `quiz_answers` .

14. Frontend

- **Shell model:** `/student/dashboard` and `/professor/dashboard` render `resources/views/app/shell.blade.php` . The shell inlines `frontend/components/navbar/navbar.html` and sets `window.LQ_HOST_CONFIG` (initialPage, role, name, avatar, preferences, idleLockMinutes). The navbar loads each page (`/pages/{role}/{page}.html`) into a **content iframe**.
- **Pages** are static HTML in `frontend/pages/{student,professor}/` . Each has a matching JS file in `frontend/assets/js/pages/{role}/` and CSS in `frontend/assets/css/pages/{role}/` . Pages call the JSON endpoints with `fetch` and send the `X-XSRF-TOKEN` header from the XSRF cookie.
- **Student pages:** home, dashboard, classwork (plus per-subject variants), enrolled-class, quiz, quest-ai, settings. **Professor pages:** home, dashboard, class, modules, module-view, competencies, worksheets (Quiz & AI Setup), quest-ai, archive, settings.
- **Shared JS** (`frontend/assets/js/common/`): dialog.js (custom alert modals, used instead of `alert()`), guided-tour.js/page-tour.js/settings-tour.js (first-run tours tracked in general_preferences), session-lock.js, insights-card.js, class-post-card.js, chart-zoom.js, page-transitions.js, frame-guard.js.
- **Blade** is used only for the landing page, auth pages (login, OTP, forgot/reset, Google role), the shell, and emails. The target structure in `CLAUDE.md` (layouts/components) is aspirational. Migrate in small steps.
- Assets are served by `/assets/{path}` → `frontend/assets` (authenticated). Auth-page assets are copied into `public/assets` .

15. Configuration reference

Key (env)	Default	Meaning
LLAMA_API_KEY / LLAMA_API_URL / LLAMA_MODEL	— / routeway v1 / llama-3.3-70b-instruct	Primary LLM (config `services.routeway`)
LLAMA_FALLBACK_MODEL / LLAMA_LAST_RESORT_MODEL	deepseek-v4-flash / deepseek-v4-flash:free	Fallback chain. Leave last resort blank to disable it.
LLAMA_API_TIMEOUT, LLAMA_MAX_RETRIES, LLAMA_RETRY_DELAY_MS	60, 2, 500	Global client limits
LLAMA_{INSIGHTS,COACH,QUIZ,COMPETENCY,SLIDE_DECK}_*	see services.php	Per-feature timeouts and retries
BKT_DEFAULT_PL0 / PT / PG / PS	0.30 / 0.15 / 0.25 / 0.10	BKT priors
BKT_THRESHOLD_LOW / HIGH	0.40 / 0.75	Mastery levels
QUIZ_DEFAULT_QUESTION_COUNT, QUIZ_QUESTIONS_PER_BATCH, QUIZ_MAX_QUESTIONS_PER_BATCH	6, 6, 15	Generation sizing
quiz.min/max_question_count, max_time_limit_minutes	5 / 15, 120	Teacher settings bounds
quiz.default_settings	mix 30/40/30, no timer, unlimited attempts, shuffle Q, show answers, adaptive on	Defaults for lessons with no saved settings
QUIZ_ADAPTIVE_SHIFT	20	Percentage points moved for low/high levels
QUIZ_TIME_LIMIT_GRACE_SECONDS	60	Late-submission grace period
QUIZ_MIN_FEEDBACK_FOR_REGENERATION	1	Gate for the regenerate action
QUIZ_TARGET_{EASY,MEDIUM,HARD}, QUIZ_CALIBRATION_TOLERANCE, QUIZ_MIN_RESPONSES_FOR_CALIBRATION	0.80/0.60/0.40, 0.15, 5	Difficulty calibration
OTP_LENGTH / EXPIRES_MINUTES / MAX_ATTEMPTS / RESEND_COOLDOWN_SECONDS	6 / 10 / 3 / 60	Email OTP
GOOGLE_CLIENT_ID / SECRET / REDIRECT_URI	blank = disabled	Google sign-in
FILESYSTEM_DISK, AWS_*	local	Storage for uploads, avatars and decks. Use a private cloud bucket in production.
LOG_AI_STACK, LOG_AI_LEVEL	ai_file, debug	AI log destination
MAIL_*	smtp.gmail.com:587	`MAIL_MAILER=log` writes emails (including OTPs) to the log in development

16. Testing

- Run `php artisan test` (or `composer test`). Tests use SQLite in memory (`phpunit.xml`). At this snapshot **315 tests pass**.
- **Unit:** BayesianKnowledgeTracingServiceTest, AdaptiveLearningServiceTest, AdaptiveQuizServiceTest, ClassInsightServiceTest, LessonContentServiceTest, LlamaServiceTest.
- **Feature:** auth (LoginOtp, LoginRemember, OtpVerification, GoogleAuth, SessionLock), classes and posts, modules and permissions, competencies and suggestions, LessonSummary, QuizGeneration, QuizRegeneration, QuizStudio,

QuizAttempt, QuizFeedback, AdaptiveQuiz, StudentMastery, dashboards and insights, QuestAi and ProfessorQuestAi, SlideDeckGeneration, Settings, AccountDeletion, SystemAlerts.

- AI calls are **never** made for real in tests. Feature tests mock or bind `LlamaService`, or use Guzzle mock handlers. Follow the same pattern and never call Routeway from tests.
- When you change BKT math, keep the hand-computed expectations in the unit test. When you add an AI feature, test its validation rules (malformed JSON, foreign competency IDs, wrong answer key) and its authorization (another professor's or student's resources return 404).

17. Coding conventions

- Follow `AGENTS.md` (Laravel Boost): PHP 8 constructor promotion, explicit return types, curly braces always, PHPDoc with array shapes, Pint formatting (`vendor/bin/pint --dirty`).
- Validate input in FormRequests (`app/Http/Requests`). Use constructor or method injection for services.
- Comments explain the *why*, often citing the research rule (e.g. "BKT computes mastery; Llama only writes replies"). Keep that tone and density.
- Controllers return JSON with a human-readable `message` on errors. Generated rows store the `model` that produced them.
- Prefer archiving or hiding to deleting when history matters (quizzes are archived, rejected questions are hidden, classes are archived).
- Do not add dependencies or new top-level folders without approval. Create docs only when asked.

18. Known gaps & gotchas

> - **PowerPoint upload is not implemented.** The spec says PDF and PPT, but uploads are validated as `mimes:pdf` only. `phpoffice/phppresentation` is used only to *write* generated decks. Adding PPT input requires a PowerPoint extractor next to `PdfTextExtractorService`.
> - **No admin area.** The `admin` role value exists, but there are no admin routes, middleware or controllers. `frontend/pages/admin/example-ui.html` is a mock-up.
> - **No queues in use.** All AI work runs synchronously in the request, bounded by timeouts and parallel batching. Moving work to queued jobs would change response contracts.
> - **Scanned PDFs** (no text layer) produce no content. OCR is out of scope. LessonContentService caches the failure as an empty string.
> - **Content cap:** 12k characters per request. Summaries use only the first 12k characters. Quizzes cover longer lessons through section-split batches.
> - **`ClassRoom::$subject` is a string**. Use `parentSubject` for the relation. Quiz generation requires `classes.subject_id`.
> - **AI artifacts attach to ClassPost**, so the same Module posted to three classes gets three independent summaries and quizzes.
> - **DB default mismatch:** `.env.example` uses sqlite while the spec says MySQL. A `learnquest.sql` dump exists at the repo root (untracked).
> - The spec lists `recommendations`, `announcements` and `audit_logs` tables. They do not exist. Recommendations are covered by AI Insights and QuestAI, and announcements are ClassPosts.
> - Grading compares the trimmed choice **text** to `correct_answer`, so the stored choices must keep the exact text.
> - Multipart updates (posts, modules) are sent as POST with `_method=PUT`.

19. Quick task recipes

Add a new AI feature

1. Add a `xxxMessages()` method to `PromptService` that requests strict JSON.

2. Create a service in `app/Services/AI` that injects `LlamaService` and `PromptService`, with its own `parseAndValidate()`. If needed, add a per-feature limits block under `services.routeway`.
3. Create a thin controller in `Http/Controllers/AI` with a FormRequest, an ownership or enrollment check, throttle middleware, and catches for the AI exceptions. Add a route in `routes/web.php` under the correct role group.
4. If mastery is needed, read it through `AdaptiveLearningService::competencyProfile`. Never ask the model for it.
5. Write feature tests with a mocked `LlamaService`, covering valid output, invalid output, and authorization.

Change adaptive behaviour

Edit the thresholds in `config/bkt.php` or the shift in `config/quiz.php`. Edit the rules only in `AdaptiveLearningService` or `AdaptiveQuizService`. Update `tests/Unit/Adaptive*Test.php`.

Add a frontend page

Add `frontend/pages/{role}/{name}.html` plus a JS and CSS file under `frontend/assets/.../pages/{role}/`, and link it from the navbar. It is then served automatically at `/pages/{role}/{name}.html` behind role middleware. Reuse `dialog.js` for modals and the shared component CSS.

Useful commands

```
composer run dev            # server + queue + pail logs + vite
php artisan test            # full suite
php artisan route:list --except-vendor
tail -f storage/logs/ai.log # every LLM request/response summary
vendor/bin/pint --dirty     # format changed files
```

Related sources in the repo: `CLAUDE.md` (specification and research intent), `docs/PROJECT_CONTEXT.md`, `AGENTS.md` (Laravel conventions), `docs/FINAL-Paper-LearnQuest-Chapter1-2.pdf` (capstone paper).